Московский государственный технический университет
имени Н. Э. Баумана

Кафедра ИУ5

Домашнее задание

Изучение языка Go
Консольный список задач

Дисциплина
Парадигмы и конструкции языков программирования

Выполнил: студент группы ИУ5-31Б
Летуновский Артемий Алексеевич

Проверил: Гапанюк Юрий Евгеньевич

Москва 2026

Задание и назначение проекта

Цель домашнего задания - изучить новый язык программирования через разработку небольшого проекта. Для работы выбран Go. Разработано консольное приложение, которое хранит список учебных задач в JSON и сохраняет данные между запусками.

Возможности приложения

Команда add добавляет задачу с непустым названием; list показывает задачи с отметками [ ] и [x]; done помечает задачу по числовому идентификатору; stats выводит общее число задач, число выполненных и оставшихся.

При первом запуске отсутствие файла означает пустой список. Для добавления и изменения задачи файл сохраняется. При чтении списка запись не выполняется. Неизвестная команда, неверный идентификатор, отсутствующая задача и поврежденный JSON приводят к сообщению об ошибке и коду завершения 1.

Средства разработки

Проверка выполнена на Windows amd64 компилятором Go 1.27.3, установленным из официального ZIP-архива. Перед распаковкой проверена контрольная сумма SHA-256. Для редактирования использовался редактор исходного кода; сборка и тесты запускались в терминале. Исходники оформлены стандартной утилитой gofmt.

Проект состоит из main.go, main_test.go и go.mod. Используется только стандартная библиотека, поэтому загрузка внешних модулей не требуется. В go.mod указана минимальная версия Go 1.22.

Границы проекта

Приложение рассчитано на последовательный запуск одним пользователем. Параллельное изменение файла не поддерживается. Проект демонстрирует базовые конструкции языка без сетевого взаимодействия и базы данных.

Конструкции языка Go в проекте

Go - компилируемый язык со статической типизацией. Программа начинается с package main, а точкой входа служит функция main. Зависимости на уровне исходного файла подключаются через import. В проекте функции отвечают за чтение, сохранение и обработку команд.

Структура Task объединяет поля ID типа int, Title типа string и Done типа bool. Экспортируемые имена с заглавной буквы доступны кодировщику JSON. Теги json задают имена id, title и done в файле. Список задач представлен срезом []Task; добавление выполняется функцией append.

Короткое объявление := выводит тип из значения. Условие if проверяет ошибки и состояние задачи. Конструкция switch выбирает обработчик команды, а цикл for с range перебирает задачи. В обработчике done перебор выполняется по индексам, чтобы менять поле Done исходного среза, а не копии элемента.

Функции могут возвращать несколько значений: loadTasks возвращает список и error. nil означает отсутствие ошибки. Ожидаемые ошибки передаются вызывающей функции и проверяются через if err != nil. Это позволяет main вывести сообщение и завершить процесс, а тестам - проверить ошибку без завершения тестового процесса.

Параметр out имеет тип интерфейса io.Writer. В обычном запуске передается os.Stdout, в тестах - bytes.Buffer. Обработчик команд не зависит от конкретного устройства вывода. Такой способ показывает использование интерфейса без отдельной иерархии классов.

Пакет encoding/json преобразует структуры в JSON и обратно. os.ReadFile и os.WriteFile работают с файлом. Пакет flag разбирает параметр -file; flag.String возвращает указатель на строку, поэтому при вызове run используется выражение *path. Пакет strconv преобразует текстовый id в целое число.

В проекте применяется процедурная организация через функции и моделирование данных структурами. Интерфейс используется для подмены вывода в тестах. Горутины и каналы не нужны, поскольку все команды выполняются последовательно.

Устройство и порядок запуска

Основные функции

loadTasks загружает список или возвращает ошибку чтения. saveTasks кодирует задачи и записывает файл. nextID ищет следующий свободный идентификатор как максимум существующих плюс один. run разбирает команду и вызывает нужные действия. main получает флаги, вызывает run и обрабатывает ошибку.

Сборка и команды

```
go version

go test -v ./...

go build -o tasker.exe .

tasker.exe add "Изучить Go"

tasker.exe list

tasker.exe done 1

tasker.exe stats
```

Команды выполняются из папки dz. В PowerShell перед именем локальной программы нужно добавить .\, например .\tasker.exe list. Без отдельной сборки можно использовать go run . list.

По умолчанию файл tasks.json создается в текущей папке. Для другого пути: go run . -file demo.json list. Флаг -file указывается перед командой. Текст задачи с пробелами удобно заключать в кавычки.

При повторном done для уже выполненной задачи состояние остается выполненным. Поврежденный JSON не перезаписывается командой add, поскольку ошибка чтения возвращается до сохранения. Ошибочный ввод также не изменяет файл.

Текст программы

main.go Часть 1 из 3

```
// Консольный список задач с сохранением в JSON.
package main

import (
    "encoding/json"
    "errors"
    "flag"
    "fmt"
    "io"
    "os"
    "strconv"
    "strings"
)

// Task объединяет данные одной задачи. Теги задают имена полей JSON.
type Task struct {
    ID    int    `json:"id"`
    Title string `json:"title"`
    Done  bool   `json:"done"`
}

// Отсутствие файла при первом запуске означает пустой список.
func loadTasks(path string) ([]Task, error) {
    data, err := os.ReadFile(path)
    if errors.Is(err, os.ErrNotExist) {
        return []Task{}, nil
    }
    if err != nil {
        return nil, err
    }
    var tasks []Task
    if err := json.Unmarshal(data, &tasks); err != nil {
        return nil, fmt.Errorf("не удалось прочитать JSON: %w", err)
    }
    return tasks, nil
}

// Кодирование выполняется до записи, чтобы проверить данные заранее.
func saveTasks(path string, tasks []Task) error {
    data, err := json.MarshalIndent(tasks, "", "  ")
    if err != nil {
        return err
    }
    return os.WriteFile(path, append(data, '\n'), 0644)
}

// Новый идентификатор больше всех уже сохраненных идентификаторов.
func nextID(tasks []Task) int {
    id := 1
    for _, task := range tasks {
```

Продолжение программы

main.go Часть 2 из 3

```
        if task.ID >= id {
            id = task.ID + 1
        }
    }
    return id
}

// run отделяет обработку команд от завершения процесса в main.
// io.Writer позволяет использовать ту же функцию в автоматических тестах.
func run(path string, args []string, out io.Writer) error {
    if len(args) == 0 {
        fmt.Fprintln(out, "Команды: add <текст> | list | done <id> | stats")
        return nil
    }
    tasks, err := loadTasks(path)
    if err != nil {
        return err
    }
    switch args[0] {
    case "add":
        // Слова команды объединяются в название; пробельный текст запрещен.
        title := strings.TrimSpace(strings.Join(args[1:], " "))
        if title == "" {
            return fmt.Errorf("укажите непустой текст задачи")
        }
        task := Task{ID: nextID(tasks), Title: title}
        if err := saveTasks(path, append(tasks, task)); err != nil {
            return err
        }
        fmt.Fprintf(out, "Добавлена задача %d: %s\n", task.ID, task.Title)
    case "list":
        // Чтение списка не создает и не изменяет файл.
        if len(args) != 1 {
            return fmt.Errorf("команда list не принимает аргументы")
        }
        if len(tasks) == 0 {
            fmt.Fprintln(out, "Список задач пуст.")
        }
        for _, task := range tasks {
            mark := " "
            if task.Done {
                mark = "x"
            }
            fmt.Fprintf(out, "%d. [%s] %s\n", task.ID, mark, task.Title)
        }
    case "done":
        // Поиск идет по ID, а не по позиции задачи в срезе.
        if len(args) != 2 {
            return fmt.Errorf("использование: done <id>")
        }
```

Продолжение программы

main.go Часть 3 из 3

```
        id, err := strconv.Atoi(args[1])
        if err != nil || id <= 0 {
            return fmt.Errorf("id должен быть положительным целым числом")
        }
        for i := range tasks {
            if tasks[i].ID == id {
                tasks[i].Done = true
                if err := saveTasks(path, tasks); err != nil {
                    return err
                }
                fmt.Fprintf(out, "Задача %d выполнена.\n", id)
                return nil
            }
        }
        return fmt.Errorf("задача %d не найдена", id)
    case "stats":
        // Подсчитываем выполненные задачи обычным циклом range.
        if len(args) != 1 {
            return fmt.Errorf("команда stats не принимает аргументы")
        }
        done := 0
        for _, task := range tasks {
            if task.Done {
                done++
            }
        }
        fmt.Fprintf(out, "Всего: %d; выполнено: %d; осталось: %d\n",
            len(tasks), done, len(tasks)-done)
    default:
        return fmt.Errorf("неизвестная команда: %s", args[0])
    }
    return nil
}

// Флаги задаются перед командой: tasker -file tasks.json list.
func main() {
    path := flag.String("file", "tasks.json", "путь к файлу задач")
    flag.Parse()
    if err := run(*path, flag.Args(), os.Stdout); err != nil {
        fmt.Fprintln(os.Stderr, "Ошибка:", err)
        os.Exit(1)
    }
}
```

Примеры выполнения

Последовательные запуски с сохранением задач в файл

```
> tasker.exe list
Список задач пуст.

> tasker.exe add "Изучить Go"
Добавлена задача 1: Изучить Go

> tasker.exe add "Подготовить домашнее задание"
Добавлена задача 2: Подготовить домашнее задание

> tasker.exe list
1. [ ] Изучить Go
2. [ ] Подготовить домашнее задание

> tasker.exe done 1
Задача 1 выполнена.

> tasker.exe list
1. [x] Изучить Go
2. [ ] Подготовить домашнее задание

> tasker.exe stats
Всего: 2; выполнено: 1; осталось: 1
```

Задачи сохраняются между отдельными запусками. После команды done 1 первая задача помечена выполненной; статистика показывает две задачи, одну выполненную и одну оставшуюся.

Проверка ошибок и файла данных

Ошибки возвращают код завершения 1

```
> tasker.exe done 99
Ошибка: задача 99 не найдена

> tasker.exe add " "
Ошибка: укажите непустой текст задачи

> tasker.exe done abc
Ошибка: id должен быть положительным целым числом
```

JSON после выполнения команд

```
> Содержимое tasks.json
[
  {
    "id": 1,
    "title": "Изучить Go",
    "done": true
  },
  {
    "id": 2,
    "title": "Подготовить домашнее задание",
    "done": false
  }
]
```

Тестирование и вывод

Версия компилятора, результаты тестов и сборки

```
> go version
go version go1.27.3 windows/amd64

> go test -v ./...
=== RUN   TestPersistence
--- PASS: TestPersistence (0.00s)
=== RUN   TestInvalidCommandDoesNotChangeFile
--- PASS: TestInvalidCommandDoesNotChangeFile (0.00s)
=== RUN   TestCorruptFileIsPreserved
--- PASS: TestCorruptFileIsPreserved (0.00s)
=== RUN   TestEmptyList
--- PASS: TestEmptyList (0.00s)
PASS
ok      github.com/artem20080205-bit/LabaASOIU/dz   0.021s

> go build -o tasker.exe .
Сборка завершилась с кодом 0.
```

В main_test.go реализованы четыре теста. TestPersistence проверяет полный цикл add, done, загрузку из файла и stats. TestInvalidCommandDoesNotChangeFile проверяет шесть ошибочных команд и неизменность файла. TestCorruptFileIsPreserved проверяет сохранность поврежденного JSON. TestEmptyList проверяет пустой список и отсутствие записи при чтении.

Все четыре теста прошли. Отдельно выполнены десять запусков собранной программы с успешными и ошибочными командами. После них содержимое JSON сверено с ожидаемыми двумя задачами. Сборка завершилась с кодом 0.

Разработанный проект показывает структуры и срезы, функции с несколькими результатами, интерфейс, обработку ошибок, ветвления, циклы и стандартные пакеты Go. Программа сохраняет список задач и корректно обрабатывает проверенные ошибки.

Источники

Условия домашнего задания
https://github.com/ugapanyuk/courses_content/wiki/DZ

Начало работы с Go
https://go.dev/doc/tutorial/getting-started

Спецификация языка Go
https://go.dev/ref/spec

Оформление программ Effective Go
https://go.dev/doc/effective_go

Документация encoding/json
https://pkg.go.dev/encoding/json

Документация flag
https://pkg.go.dev/flag

Документация testing
https://pkg.go.dev/testing